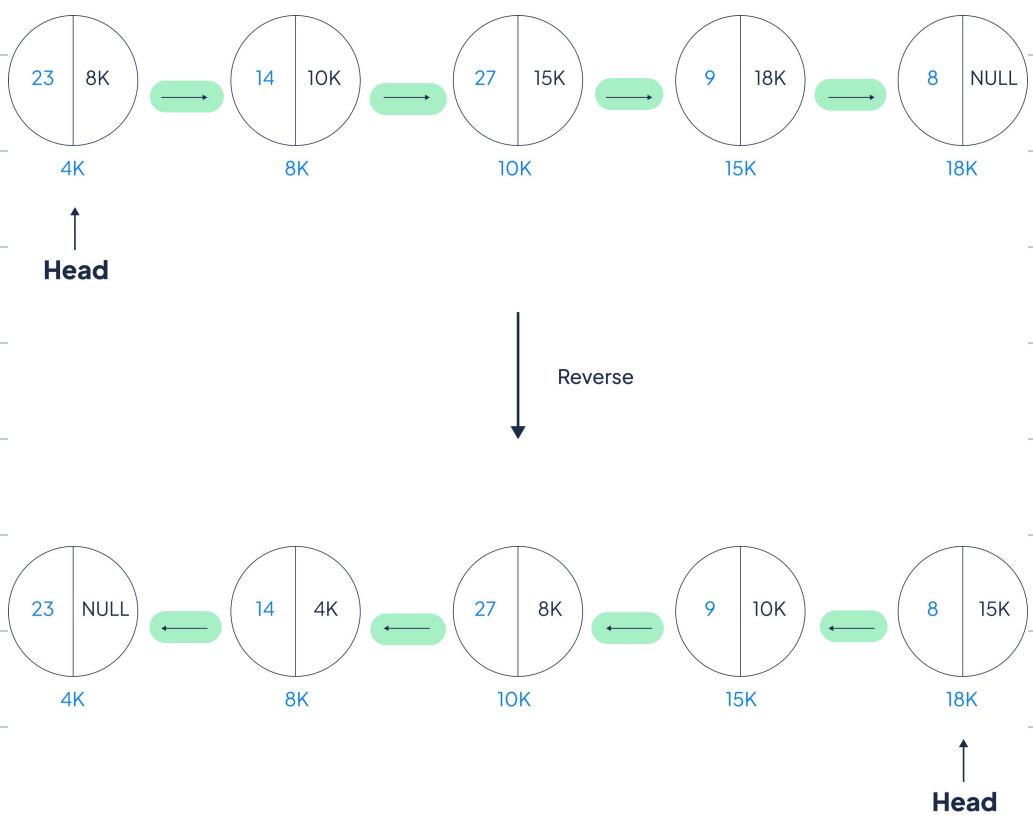
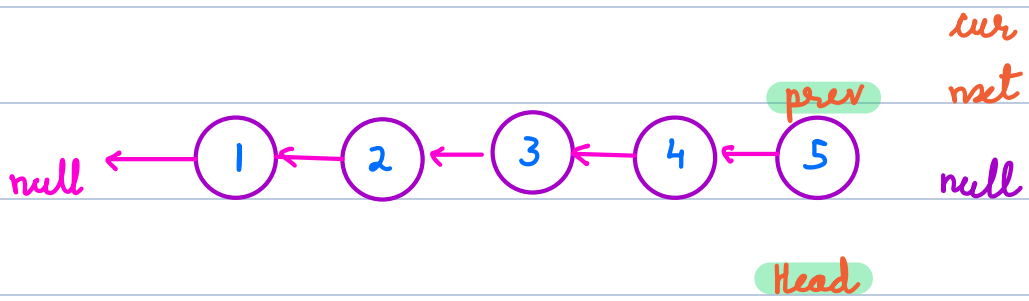
Agenda:

1. Reverse the Linked List
2. Reverse the Linked List using Recursion
3. Shallow Copy vs Deep Copy
4. Deep Copy of Linked List
5. Middle Element of List

Reverse the Linked List

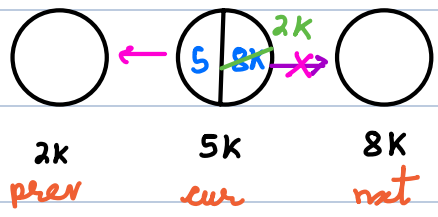
Q1: Given a linked list, **reverse the link** of each node of the linked list. [**SC = O(1)**]





$\boxed{cur.next = prev}$

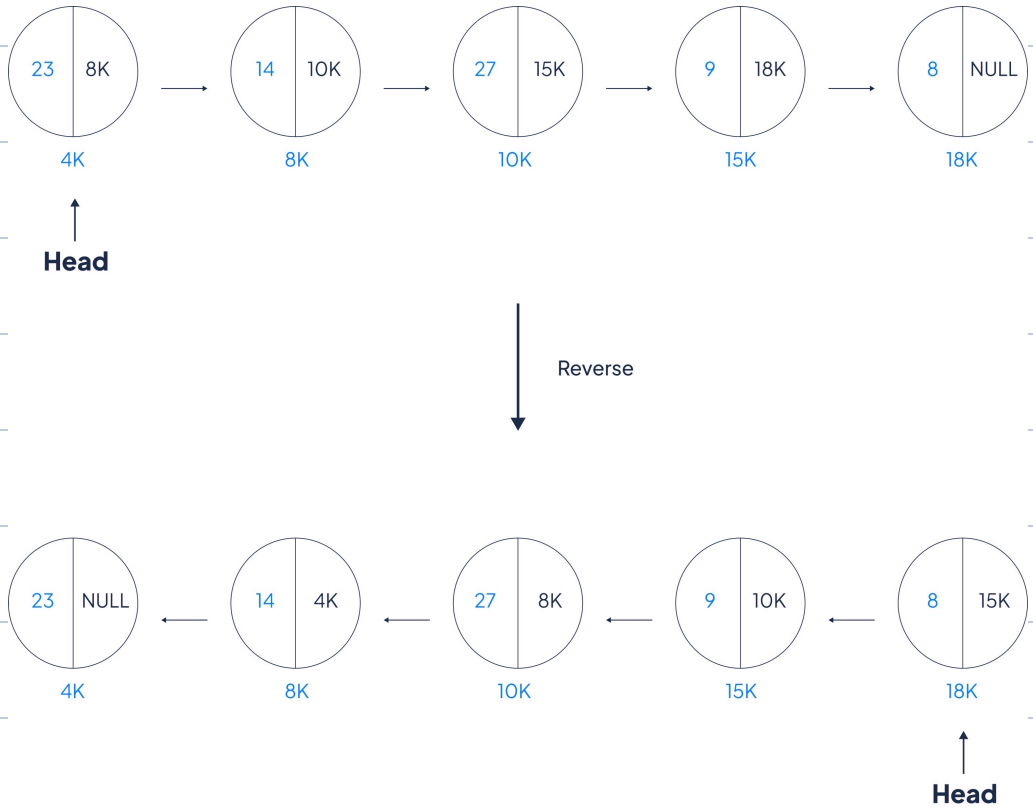
```
cur = Head        prev = null
while ( cur != null ) {
    nxt = cur.next
    cur.next = prev
    prev = cur
    cur = nxt
}
Head = prev
```

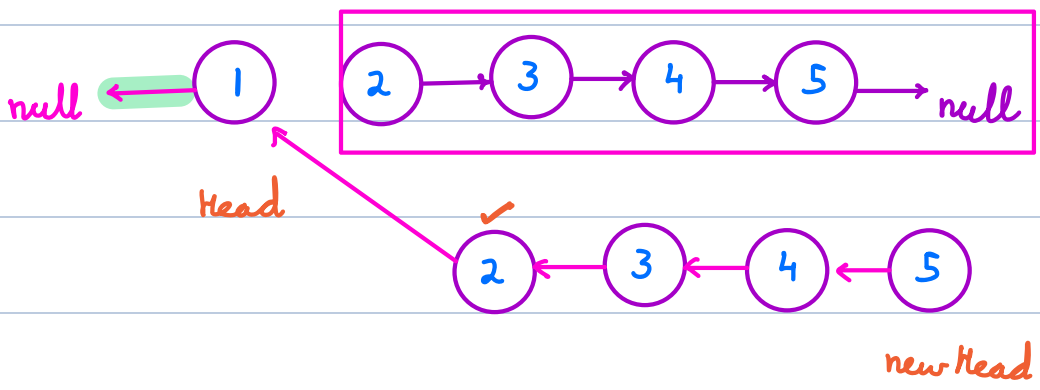


TC = O(N) SC = O(1)

Reverse the Linked List using Recursion

Q2: Given a linked list, **reverse the link** of each node of the linked list using recursion.

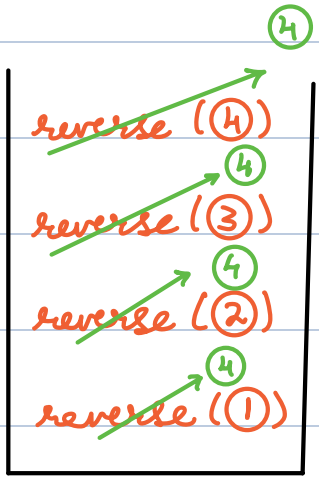




Head.next.next = Head

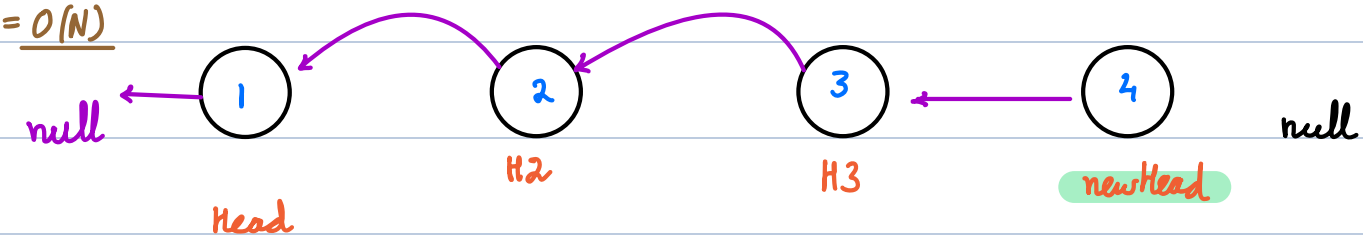
Head.next = null

```
Node reverse ( Node Head ) {
    if ( Head == null || Head.next == null)
        return Head
    Node newHead = reverse (Head.next)
    Head.next.next = Head
    Head.next = null
    return newHead
```



TC = O(N)

SC = O(N)



Shallow Copy vs Deep Copy

✅ Shallow Copy

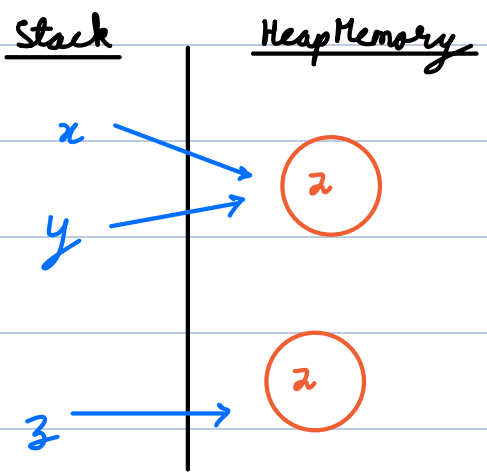
Copies **only the reference**

→ *Both variables point to the same linked list in memory.*

✅ Deep Copy

Creates a **brand new linked list** with **new nodes**, same values

→ *Two independent linked lists in memory.*



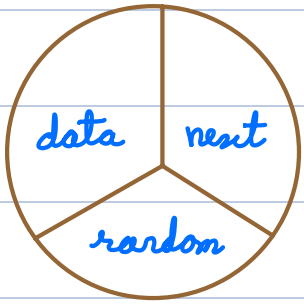

```
x = y   // shallow copy

z = new Node(x.data)
    // deep copy
```

Feature	Shallow Copy	Deep Copy
Nodes created	0	New nodes created for all elements
Memory	Shared	Independent
Safe to modify?	❌ No	✓ Yes
Complexity	O(1)	O(N) → N = # items in object
Use Case	Temporary alias	Safe duplication / algorithms

Deep Copy of Linked List

Q3: You are given a **special** Linked List where each node has:

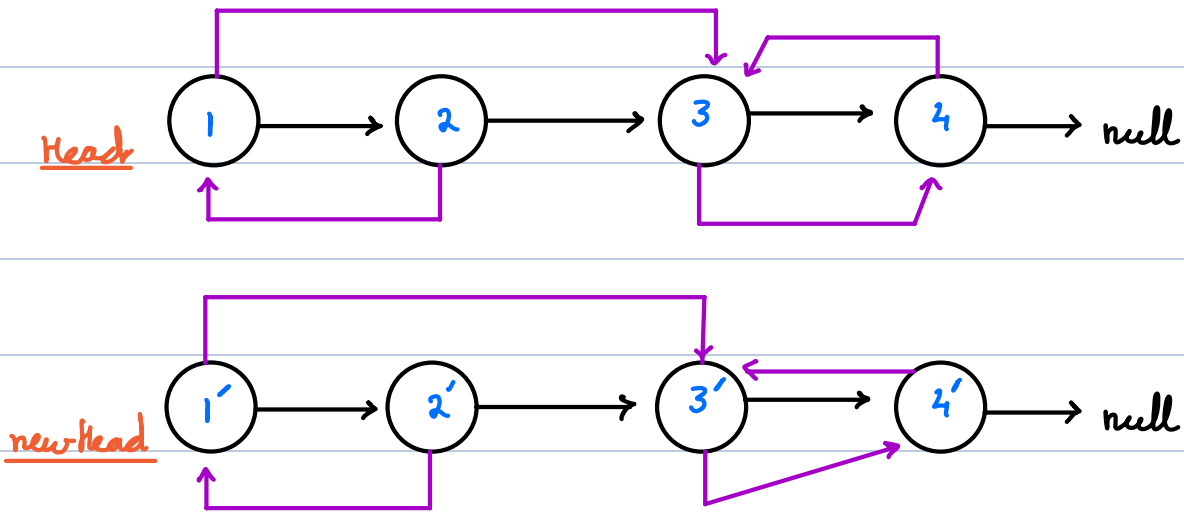
- **next** → points to the next node ✓
- **random** → points to any node in the list ✓



Your task is to **create a Deep Copy** of the given linked list with:

✓ New nodes ✓ Same values ✓ Same random-pointer structure

❌ But NO link should point back to original list




```
newHead = new Node(Head.data)
t1 = Head
t2 = newHead          hm.put(t1, t2)
while (t1.next != null) {
    t1 = t1.next
    t2.next = new Node(t1.data)
    t2 = t2.next
```

HashMap (hm)

1 → 1'

2 → 2'

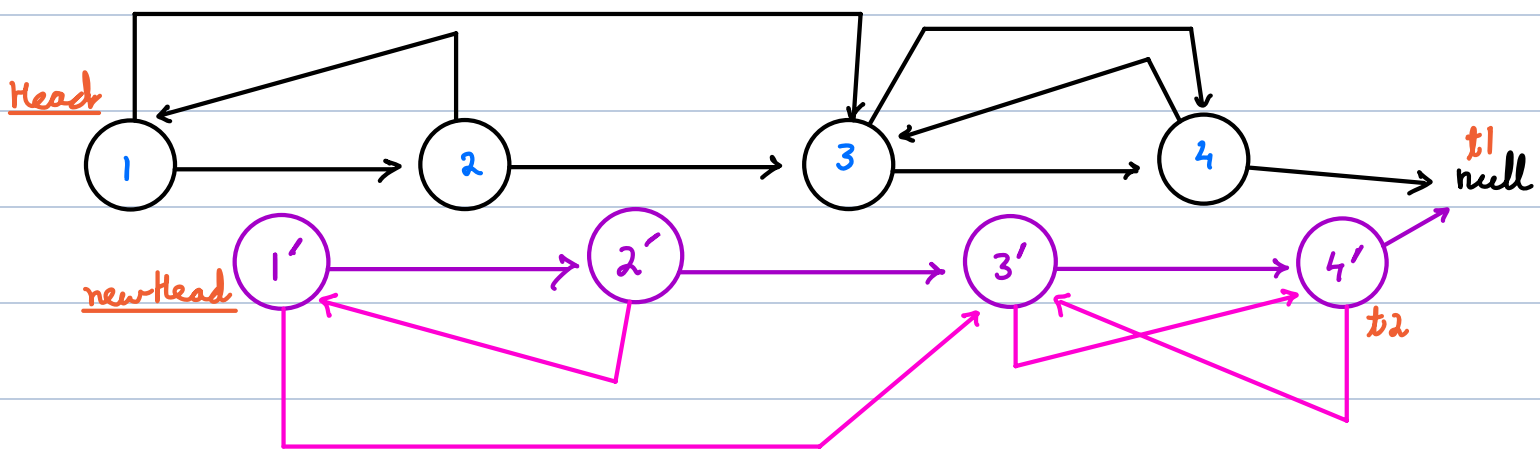
3 → 3'

4 → 4'

```
        hm.put(t1, t2)
}
```

```
t1 = Head                    t2 = newHead
while (t1 != null) {
        t2.random = hm.get(t1.random)
        t1 = t1.next
        t2 = t2.next
}
```

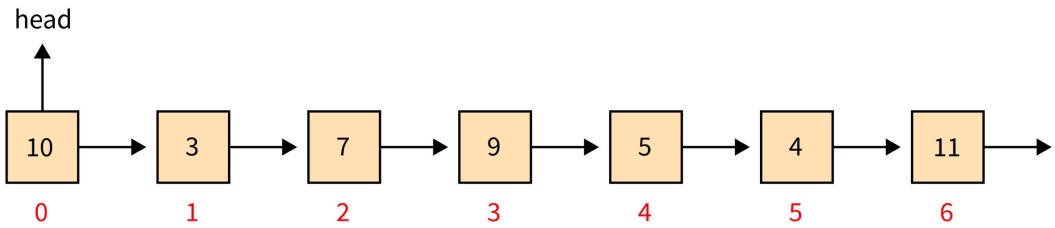
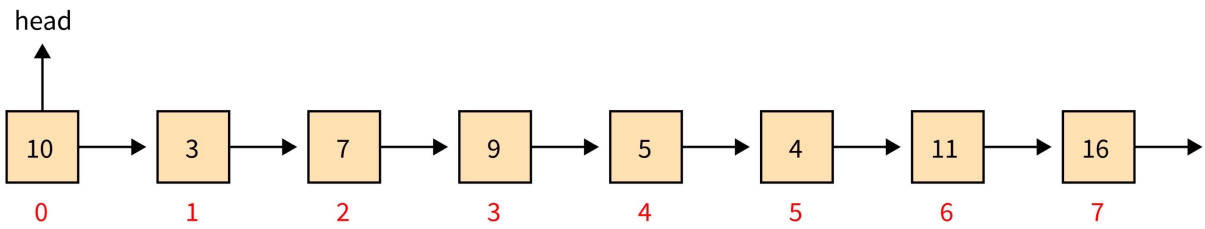
TC = O(N) SC = O(N)




```
newHead = new Node(Head.data)
newHead.next = Head.next
Head.next = newHead
t1 = Head                    t2 = newHead
while (t2.next != null) {
        t1 = t2.next
```

```
        t2 = new Node(t1.data)
        t2.next = t1.next
        t1.next = t2
    }

    t1 = Head            t2 = newHead
    while (t1 != null) {
        t2.random = t1.random.next
        t1 = t2.next
        if (t1 != null)  t2 = t1.next
    }


    t1 = Head            t2 = newHead
    while (t1 != null) {
        t1.next = t2.next
        t1 = t1.next
        if (t1 != null) { t2.next = t1.next
                          t2 = t2.next }
    }

    return newHead
```

TC = O(N) SC = O(1)

Middle Element of List

Q4: Find the middle element of the given linked list.